

Studentų dienynas

Generated by Doxygen 1.9.8

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Studentas Class Reference	7
4.2 Zmogus Class Reference	7
4.2.1 Detailed Description	8
4.2.2 Constructor & Destructor Documentation	8
4.2.2.1 Zmogus() [1/3]	8
4.2.2.2 ~Zmogus()	9
4.2.2.3 Zmogus() [2/3]	9
4.2.2.4 Zmogus() [3/3]	9
4.2.3 Member Function Documentation	9
4.2.3.1 getPavarde()	9
4.2.3.2 getVardas()	9
4.2.3.3 operator=() [1/2]	10
4.2.3.4 operator=() [2/2]	10
4.2.3.5 print()	10
4.2.3.6 setPavarde()	10
4.2.3.7 setVardas()	11
5 File Documentation	13
5.1 funkcijos.h	13
5.2 funkcijos_temp.h	14
5.3 grupavimas.h	17
5.4 lib.h	17
5.5 rusiavimas.h	18
5.6 studentas.h	19
5.7 testavimas.h	21
Index	23

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	7
Studentas	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	Represents a student with homework grades, an exam result, and computed final scores . . .	7
Zmogus	Abstract base class representing a person with a first and last name	7

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/funkcijos.h	13
include/funkcijos_temp.h	14
include/grupavimas.h	17
include/lib.h	17
include/rusiavimas.h	18
include/studentas.h	19
include/testavimas.h	21

Chapter 4

Class Documentation

4.1 Studentas Class Reference

Represents a student with homework grades, an exam result, and computed final scores.

```
#include <studentas.h>
```

Inheritance diagram for Studentas:

4.2 Zmogus Class Reference

Abstract base class representing a person with a first and last name.

```
#include <studentas.h>
```

Inheritance diagram for Zmogus:

Collaboration diagram for Zmogus:

Public Member Functions

- **Zmogus** ()
Default constructor. Initialises name and surname to empty strings.
- **Zmogus** (const std::string &vardas, const std::string &pavarde)
Parameterised constructor.
- std::string **getVardas** () const
Returns the first name.
- std::string **getPavarde** () const
Returns the surname.
- void **setVardas** (const std::string &vardas)
Sets the first name.
- void **setPavarde** (const std::string &pavarde)
Sets the surname.
- virtual void **print** () const =0
Pure virtual function that prints information about the person.

- virtual `~Zmogus()`
Virtual destructor. Clears name fields before destruction.
- `Zmogus(const Zmogus &other)`
Copy constructor. Creates a deep copy of another Zmogus.
- `Zmogus & operator= (const Zmogus &other)`
Copy assignment operator.
- `Zmogus(Zmogus &&other) noexcept`
Move constructor. Transfers ownership of name strings.
- `Zmogus & operator= (Zmogus &&other) noexcept`
Move assignment operator.

Protected Attributes

- `std::string Vardas_`
First name of the person.
- `std::string Pavarde_`
Last name (surname) of the person.

4.2.1 Detailed Description

Abstract base class representing a person with a first and last name.

`Zmogus` (Lithuanian: "person") serves as an abstract base class in the class hierarchy. It stores a person's name and surname and enforces a pure virtual `print()` function that all derived classes must implement.

It fully satisfies the Rule of Five:

- Default and parameterised constructors
- Copy constructor and copy assignment operator
- Move constructor and move assignment operator
- Virtual destructor (required for safe polymorphic deletion)

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Zmogus() [1/3]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde ) [inline]
```

Parameterised constructor.

Parameters

<code>vardas</code>	First name of the person.
<code>pavarde</code>	Surname of the person.

4.2.2.2 ~Zmogus()

```
virtual Zmogus::~~Zmogus ( ) [inline], [virtual]
```

Virtual destructor. Clears name fields before destruction.

The destructor is virtual so that deleting a `Zmogus*` that actually points to a `Studentas` will correctly call `Studentas`'s destructor.

4.2.2.3 Zmogus() [2/3]

```
Zmogus::Zmogus (
    const Zmogus & other ) [inline]
```

Copy constructor. Creates a deep copy of another `Zmogus`.

Parameters

<i>other</i>	The <code>Zmogus</code> instance to copy from.
--------------	--

4.2.2.4 Zmogus() [3/3]

```
Zmogus::Zmogus (
    Zmogus && other ) [inline], [noexcept]
```

Move constructor. Transfers ownership of name strings.

Parameters

<i>other</i>	The <code>Zmogus</code> instance to move from. Left in a valid empty state.
--------------	---

4.2.3 Member Function Documentation

4.2.3.1 getPavarde()

```
std::string Zmogus::getPavarde ( ) const [inline]
```

Returns the surname.

Returns

Surname as a string.

4.2.3.2 getVardas()

```
std::string Zmogus::getVardas ( ) const [inline]
```

Returns the first name.

Returns

First name as a string.

4.2.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & other ) [inline]
```

Copy assignment operator.

Parameters

<i>other</i>	The Zmogus instance to copy from.
--------------	---

Returns

Reference to this object.

4.2.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && other ) [inline], [noexcept]
```

Move assignment operator.

Parameters

<i>other</i>	The Zmogus instance to move from. Left in a valid empty state.
--------------	--

Returns

Reference to this object.

4.2.3.5 print()

```
virtual void Zmogus::print ( ) const [pure virtual]
```

Pure virtual function that prints information about the person.

Every derived class must provide its own implementation. This makes [Zmogus](#) an abstract class — you cannot create a plain [Zmogus](#) object directly.

Implemented in [Studentas](#).

4.2.3.6 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & pavarde ) [inline]
```

Sets the surname.

Parameters

<i>pavarde</i>	New surname.
----------------	--------------

4.2.3.7 setVardas()

```
void Zmogus::setVardas (
    const std::string & vardas ) [inline]
```

Sets the first name.

Parameters

<i>vardas</i>	New first name.
---------------	-----------------

The documentation for this class was generated from the following file:

- include/studentas.h

Chapter 5

File Documentation

5.1 funkcijos.h

```
00001 #ifndef FUNKCIJOS_H
00002 #define FUNKCIJOS_H
00003
00004 #include "lib.h"
00005 #include "studentas.h"
00006
00007 bool ar_valid (const string &input);
00008 int meniu();
00009 int mok_sk_ivedimas();
00010 void pazymiu_ivedimas(Studentas &s);
00011 int egz_ivedimas();
00012 double vidurkis(Studentas &s, int paz_sk);
00013 template<typename Container>
00014 void mokinio_info_ivedimas(Container &S, int mok_sk);
00015 int paz_sk_ivedimas();
00016 string med_ar_vid();
00017 template<typename Container>
00018 void pasirinkimas2(Container &S, int meniu_pasirinkimas);
00019 template<typename Container>
00020 void pasirinkimas3(Container &S, int meniu_pasirinkimas);
00021 string rnd_vardas(bool vyras);
00022 string rnd_pavarde(bool vyras);
00023 void failo_generavimas();
00024 template<typename Container>
00025 void ivedimas_is_failo(Container &S, int meniu_pasirinkimas);
00026 template<typename Container>
00027 void rusiavimas(Container &S, int did_ar_maz, int pagal_ka_rusiuoti);
00028 template<typename Container>
00029 void isvedimas_i_faila(const Container &Dundukai, const Container &Galvociai);
00030 void test_failu_generavimas();
00031 template<typename Container>
00032 void antras_tyrimas();
00033 template<typename Container>
00034 void rusiavimo_pasirinkimas(Container &S);
00035 template<typename Container>
00036 void rasyti_i_faila(const Container& grupe, const std::string& failo_pavadinimas);
00037 template<typename Container>
00038 void grupavimas_pirm(Container &S, Container &Dundukai, Container &Galvociai);
00039 template<typename Container>
00040 void grupavimas_antras(Container &S, Container &Dundukai);
00041 template<typename Container>
00042 void nuskaitymas(Container &S, const int& dydis);
00043 template<typename Container>
00044 void skaiciavimai(Container &S);
00045 template<typename Container>
00046 void grupavimas_trecias(Container& S, Container& Dundukai);
00047
00048 #include "funkcijos_temp.h"
00049 #include "rusiavimas.h"
00050 #include "testavimas.h"
00051 #include "grupavimas.h"
00052
00053 #endif
```

5.2 funkcijos_temp.h

```

00001 #ifndef FUNKCIJOS_TEMP_H
00002 #define FUNKCIJOS_TEMP_H
00003
00004 #include "lib.h"
00005 #include "studentas.h"
00006
00025 template<typename Container>
00026 void mokinio_info_ivedimas(Container &S, int mok_sk) {
00027     int i = 0;
00028     std::string eil, vrd, pvrđ;
00029     while (true) {
00030         if (mok_sk != 0 && i >= mok_sk) break;
00031
00032         std::cout << "Iveskite " << i + 1 << "-ojo mokinio varda ir pavarde. Tuscia eilute - baigti\n";
00033         std::getline(std::cin, eil);
00034
00035         if (mok_sk == 0 && eil.empty() && i != 0) break;
00036
00037         std::stringstream ss(eil);
00038         if (!(ss >> vrd >> pvrđ)) {
00039             std::cout << "Klaida! Turite ivesti varda ir pavarde...\n";
00040             std::cin.clear();
00041             continue;
00042         }
00043
00044         S.push_back(Studentas(vrd, pvrđ));
00045         int paz_sk;
00046         pazymiu_ivedimas(S.back());
00047         S.back().setVidurkis(vidurkis(S.back(), paz_sk));
00048         S.back().setEgzRez(egz_ivedimas());
00049         i++;
00050     }
00051 }
00052
00077 template<typename Container>
00078 void ivedimas_is_failo(Container &S, int meniu_pasirinkimas) {
00079     string path;
00080     cout << "Iveskite failo, is kurio norite nuskaityti, pavadinima (pvz., kursiokai.txt): ";
00081     cin >> path;
00082     if (path.empty() || path.size() < 4 || path.substr(path.size() - 4) != ".txt") {
00083         cin.clear();
00084         throw std::runtime_error("Klaida! Failo pavadinimas turi baigtis .txt...\n");
00085     }
00086     std::ifstream in(path);
00087     if (!in) {
00088         throw std::runtime_error("Klaida! Nepavyko atidaryti failo...\n");
00089     }
00090     string eil;
00091     getline(in, eil); // skip header
00092     while (getline(in, eil)) {
00093         int paz_suma = 0;
00094         if (eil.empty()) continue;
00095
00096         std::stringstream ss(eil);
00097         int paz;
00098         Studentas tmp;
00099         string vrd, pvrđ;
00100         ss >> vrd >> pvrđ;
00101         tmp.setVardas(vrd);
00102         tmp.setPavarde(pvrđ);
00103         if (meniu_pasirinkimas == 1) {
00104             while (ss >> paz) {
00105                 if (paz > 0 && paz <= 10) {
00106                     tmp.getNdRez().push_back(paz);
00107                     paz_suma += paz;
00108                 } else {
00109                     throw std::runtime_error("Klaida! Pazymiai turi buti desimtbałeje
sistemoje...\n");
00110                 }
00111             }
00112         }
00113         if (!tmp.getNdRez().empty()) {
00114             tmp.setEgzRez(tmp.getNdRez().back());
00115             tmp.getNdRez().pop_back();
00116         } else if (meniu_pasirinkimas == 1) {
00117             throw std::runtime_error("Klaida! Mokinio " + tmp.getVardas() + " "
+ tmp.getPavarde() + " namu darbu pazymiai nerasti...\n");
00118         }
00119         tmp.setVidurkis(static_cast<double>(paz_suma - tmp.getEgzRez()))
/ static_cast<double>(tmp.getNdRez().size());
00120         S.push_back(tmp);
00121     }
00122 }
00123
00124 }
00125
00137 template<typename Container>

```

```

00138 void rasyti_i_faila(const Container& grupe, const std::string& failo_pavadinimas) {
00139     std::ofstream out(failo_pavadinimas);
00140     if (!out) {
00141         std::cerr << "Klaida! Nepavyko sukurti failo " << failo_pavadinimas << "...\\n";
00142         return;
00143     }
00144     out << std::fixed << std::setprecision(2);
00145     out << std::left
00146         << std::setw(15) << "Vardas"
00147         << std::setw(25) << "Pavarde"
00148         << std::setw(18) << "Galutinis (Vid.)"
00149         << std::setw(18) << "Galutinis (Med.)" << '\\n';
00150     out << std::string(76, '-') << '\\n';
00151     for (const auto& s : grupe) {
00152         out << std::left
00153             << std::setw(15) << s.getVardas()
00154             << std::setw(25) << s.getPavarde()
00155             << std::setw(18) << s.getGalutinisVid()
00156             << std::setw(18) << s.getGalutinisMed()
00157             << '\\n';
00158     }
00159 }
00160
00172 template<typename Container>
00173 void isvedimas_i_faila(const Container &Dundukai, const Container &Galvociai) {
00174     rasyti_i_faila(Dundukai, "dundukai.txt");
00175     rasyti_i_faila(Galvociai, "galvociai.txt");
00176 }
00177
00187 template<typename Container>
00188 void isvedimas(Container S) {
00189     std::cout << std::left
00190         << std::setw(15) << "Vardas"
00191         << std::setw(25) << "Pavarde"
00192         << std::setw(18) << "Galutinis (Vid.)"
00193         << std::setw(18) << "Galutinis (Med.)\\n";
00194     for (int i = 0; i < 76; i++) std::cout << "-";
00195     std::cout << std::endl;
00196     for (auto& s : S) {
00197         std::cout << std::left
00198             << std::setw(15) << s.getVardas()
00199             << std::setw(25) << s.getPavarde()
00200             << std::setw(18) << std::fixed << std::setprecision(2) << s.getGalutinisVid()
00201             << std::setw(18) << std::fixed << std::setprecision(2) << s.getGalutinisMed()
00202             << std::endl;
00203     }
00204 }
00205
00223 template<typename Container>
00224 void pasirinkimas2(Container &S, int meniu_pasirinkimas) {
00225     int mok_sk;
00226     std::cout << "0 - ivedimas ranka, 1 - ivedimas is failo\\n";
00227     int input;
00228     try {
00229         std::cin >> input;
00230         std::cin.ignore(10000, '\\n');
00231         if (!std::cin || (input != 0 && input != 1)) {
00232             throw std::runtime_error("Klaida! Turite ivesti 0 arba 1...\\n");
00233         }
00234     }
00235     catch (const std::exception& e) {
00236         std::cerr << e.what();
00237         std::cin.clear();
00238         std::cin.ignore(10000, '\\n');
00239         return;
00240     }
00241     bool ar_is_failo = (input == 1);
00242     if (ar_is_failo) {
00243         try {
00244             ivedimas_is_failo(S, meniu_pasirinkimas);
00245             for (auto& s : S) s.getNdRez().clear();
00246         }
00247         catch (const std::exception& e) {
00248             std::cerr << e.what();
00249             return;
00250         }
00251     } else {
00252         mok_sk = mok_sk_ivedimas();
00253     }
00254
00255     int i = 0;
00256     std::string eil, vrd, pvrđ;
00257     while (true) {
00258         if (ar_is_failo || (mok_sk != 0 && i >= mok_sk)) break;
00259
00260         std::cout << "Iveskite " << i + 1 << "-ojo mokinio varda ir pavarde. Tuscia eilute - baigti\\n";
00261         std::getline(std::cin, eil);

```

```

00262
00263         if (mok_sk == 0 && eil.empty() && i != 0) break;
00264
00265         std::stringstream ss(eil);
00266         if (!(ss >> vrd >> pvrđ)) {
00267             std::cerr << "Klaida! Turite ivesti varda ir pavarde...\n";
00268             std::cin.clear();
00269             continue;
00270         }
00271         S.push_back(Studentas(vrd, pvrđ));
00272         i++;
00273     }
00274
00275     std::cout << "Generuojami pazymiai...\n";
00276     for (auto& s : S) {
00277         int n = rand() % 100 + 1;
00278         for (int i = 0; i < n; i++) s.getNdRez().push_back(rand() % 10 + 1);
00279         s.setEgzRez(rand() % 10 + 1);
00280         s.setVidurkis(vidurkis(s, s.getNdRez().size()));
00281     }
00282     for (auto& s : S) {
00283         std::sort(s.getNdRez().begin(), s.getNdRez().end());
00284         if (s.getNdRez().size() % 2 == 0) {
00285             s.setMediana((s.getNdRez()[s.getNdRez().size() / 2]
00286                 + s.getNdRez()[s.getNdRez().size() / 2 - 1]) / 2.0);
00287         } else {
00288             s.setMediana(s.getNdRez()[s.getNdRez().size() / 2]);
00289         }
00290         s.setGalutinisVid(0.4 * s.getVidurkis() + 0.6 * static_cast<double>(s.getEgzRez()));
00291         s.setGalutinisMed(0.4 * s.getMediana() + 0.6 * static_cast<double>(s.getEgzRez()));
00292     }
00293
00294     rusiavimo_pasirinkimas(S);
00295     Container Dundukai, Galvociai;
00296     for (auto& s : S) {
00297         if (s.getGalutinisVid() >= 5.0) Galvociai.push_back(s);
00298         else Dundukai.push_back(s);
00299     }
00300
00301     std::cout << "1 - isvesti i konsole, 2 - isvesti i faila\n";
00302     int kur_isvesti;
00303     try {
00304         std::cin >> kur_isvesti;
00305         std::cin.ignore(10000, '\n');
00306         if (!std::cin || (kur_isvesti != 1 && kur_isvesti != 2)) {
00307             throw std::runtime_error("Klaida! Neteisingai ivedete rezultatu isvedimo buda...\n");
00308         }
00309     }
00310     catch (const std::exception& e) {
00311         std::cerr << e.what();
00312         std::cin.clear();
00313         std::cin.ignore(10000, '\n');
00314         return;
00315     }
00316     if (kur_isvesti == 1) isvedimas(S);
00317     else isvedimas_i_faila(Dundukai, Galvociai);
00318 }
00319
00335 template<typename Container>
00336 void pasirinkimas3(Container &S, int meniu_pasirinkimas) {
00337     std::cout << "Generuojami studentu vardai ir pavardes...\n";
00338     int mok_sk = rand() % 100 + 1;
00339     for (int i = 0; i < mok_sk; i++) {
00340         bool vyras = rand() % 2;
00341         string vardas = rnd_vardas(vyras);
00342         string pavarde = rnd_pavarde(vyras);
00343         S.push_back(Studentas(vardas, pavarde));
00344         int n = rand() % 100 + 1;
00345         for (int j = 0; j < n; j++) S.back().getNdRez().push_back(rand() % 10 + 1);
00346         S.back().setEgzRez(rand() % 10 + 1);
00347         int paz_sk = S.back().getNdRez().size();
00348         S.back().setVidurkis(vidurkis(S.back(), paz_sk));
00349         sort(S.back().getNdRez().begin(), S.back().getNdRez().end());
00350         if (S.back().getNdRez().size() % 2 == 0) {
00351             S.back().setMediana((S.back().getNdRez()[S.back().getNdRez().size() / 2]
00352                 + S.back().getNdRez()[S.back().getNdRez().size() / 2 - 1]) / 2.0);
00353         } else {
00354             S.back().setMediana(S.back().getNdRez()[S.back().getNdRez().size() / 2]);
00355         }
00356         S.back().setGalutinisVid(0.4 * S.back().getVidurkis()
00357             + 0.6 * static_cast<double>(S.back().getEgzRez()));
00358         S.back().setGalutinisMed(0.4 * S.back().getMediana()
00359             + 0.6 * static_cast<double>(S.back().getEgzRez()));
00360     }
00361
00362     rusiavimo_pasirinkimas(S);
00363     Container Dundukai, Galvociai;

```

```

00364     for (auto& s : S) {
00365         if (s.getGalutinisVid() >= 5.0) Galvociai.push_back(s);
00366         else Dundukai.push_back(s);
00367     }
00368
00369     std::cout << "1 - isvesti i konsolę, 2 - isvesti i failą\n";
00370     int kur_isvesti;
00371     try {
00372         std::cin >> kur_isvesti;
00373         std::cin.ignore(10000, '\n');
00374         if (!std::cin || (kur_isvesti != 1 && kur_isvesti != 2)) {
00375             throw std::runtime_error("Klaida! Neteisingai ivedete rezultatu isvedimo buda...\n");
00376         }
00377     }
00378     catch (const std::exception& e) {
00379         std::cerr << e.what();
00380         std::cin.clear();
00381         std::cin.ignore(10000, '\n');
00382         return;
00383     }
00384     if (kur_isvesti == 1) isvedimas(S);
00385     else isvedimas_i_faila(Dundukai, Galvociai);
00386 }
00387
00388 #endif

```

5.3 grupavimas.h

```

00001 #ifndef GRUPAVIMAS_H
00002 #define GRUPAVIMAS_H
00003
00004 #include "lib.h"
00005 #include "studentas.h"
00006
00019 template<typename Container>
00020 void grupavimas_pirm(Container &S, Container &Dundukai, Container &Galvociai) {
00021     for (auto& s : S) {
00022         if (s.getGalutinisVid() >= 5.0) {
00023             Galvociai.push_back(s);
00024         } else {
00025             Dundukai.push_back(s);
00026         }
00027     }
00028 }
00029
00044 template<typename Container>
00045 void grupavimas_antras(Container& S, Container& Dundukai) {
00046     rusiavimas(S, 2, 3); // sort descending by galutinis_vid_
00047     while (S.back().getGalutinisVid() < 5) {
00048         Dundukai.push_back(S.back());
00049         S.pop_back();
00050     }
00051 }
00052
00070 template<typename Container>
00071 void grupavimas_trecias(Container& S, Container& Dundukai) {
00072     auto riba = std::stable_partition(S.begin(), S.end(), [](const Studentas& s) {
00073         return s.getGalutinisVid() >= 5.0;
00074     });
00075     Dundukai.insert(Dundukai.end(), riba, S.end());
00076     S.erase(riba, S.end());
00077 }
00078
00079 #endif

```

5.4 lib.h

```

00001 #ifndef LIB_H
00002 #define LIB_H
00003
00004 #include <iostream>
00005 #include <vector>
00006 #include <string>
00007 #include <iomanip>
00008 #include <algorithm>
00009 #include <sstream>
00010 #include <fstream>
00011 #include <cstdlib>
00012 #include <list>

```

```

00013 #include <deque>
00014 #include <chrono>
00015
00016 using std::string;
00017 using std::vector;
00018 using std::cout;
00019 using std::cin;
00020 using std::stringstream;
00021 using std::getline;
00022
00023 #endif

```

5.5 rusiavimas.h

```

00001 #ifndef RUSIAVIMAS_H
00002 #define RUSIAVIMAS_H
00003
00004 #include "funkcijos.h"
00005 #include "lib.h"
00006
00022 template<typename Container>
00023 void rusiavimo_pasirinkimas(Container &S) {
00024     std::cout << "1 - rusiuoti didejanciai, 2 - rusiuoti mazejanciai\n";
00025     int did_ar_maz;
00026     try {
00027         std::cin >> did_ar_maz;
00028         std::cin.ignore(10000, '\n');
00029         if (!std::cin || (did_ar_maz != 1 && did_ar_maz != 2)) {
00030             throw std::runtime_error("Klaida! Neteisingai pasirinktas rusiavimas...\n");
00031         }
00032     }
00033     catch (const std::exception& e) {
00034         std::cerr << e.what();
00035         std::cin.clear();
00036         std::cin.ignore(10000, '\n');
00037         return;
00038     }
00039
00040     int pagal_ka_rusiuoti;
00041     std::cout << "1 - rusiuoti pagal vardą, 2 - rusiuoti pagal pavardę, "
00042         << "3 - rusiuoti pagal galutini (Vid.) rezultata, "
00043         << "4 - rusiuoti pagal galutini (Med.) rezultata\n";
00044     try {
00045         std::cin >> pagal_ka_rusiuoti;
00046         std::cin.ignore(10000, '\n');
00047         if (!std::cin || pagal_ka_rusiuoti < 1 || pagal_ka_rusiuoti > 4) {
00048             throw std::runtime_error("Klaida! Neteisingai pasirinkta pagal ka rusiuoti...\n");
00049         }
00050     }
00051     catch (const std::exception& e) {
00052         std::cerr << e.what();
00053         std::cin.clear();
00054         std::cin.ignore(10000, '\n');
00055         return;
00056     }
00057
00058     rusiavimas(S, did_ar_maz, pagal_ka_rusiuoti);
00059 }
00060
00079 template<typename Container>
00080 void rusiavimas(Container &S, int did_ar_maz, int pagal_ka_rusiuoti) {
00081     if (did_ar_maz == 1) {
00082         switch (pagal_ka_rusiuoti) {
00083             case 1:
00084                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00085                     return a.getVardas() < b.getVardas();
00086                 });
00087                 break;
00088             case 2:
00089                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00090                     return a.getPavarde() < b.getPavarde();
00091                 });
00092                 break;
00093             case 3:
00094                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00095                     return a.getGalutinisVid() < b.getGalutinisVid();
00096                 });
00097                 break;
00098             case 4:
00099                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00100                     return a.getGalutinisMed() < b.getGalutinisMed();
00101                 });
00102                 break;

```

```

00103     }
00104     } else {
00105         switch (pagal_ka_rusiuoti) {
00106             case 1:
00107                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00108                     return a.getVardas() > b.getVardas();
00109                 });
00110                 break;
00111             case 2:
00112                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00113                     return a.getPavarde() > b.getPavarde();
00114                 });
00115                 break;
00116             case 3:
00117                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00118                     return a.getGalutinisVid() > b.getGalutinisVid();
00119                 });
00120                 break;
00121             case 4:
00122                 std::sort(S.begin(), S.end(), [](const Studentas& a, const Studentas& b) {
00123                     return a.getGalutinisMed() > b.getGalutinisMed();
00124                 });
00125                 break;
00126         }
00127     }
00128 }
00129
00130 #endif

```

5.6 studentas.h

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "lib.h"
00005
00019 class Zmogus {
00020     protected:
00021         std::string Vardas_;
00022         std::string Pavarde_;
00023
00024     public:
00025
00026
00030         Zmogus()
00031             : Vardas_(""), Pavarde_("") {}
00032
00038         Zmogus(const std::string& vardas, const std::string& pavarde)
00039             : Vardas_(vardas), Pavarde_(pavarde) {}
00040
00041
00042
00047         std::string getVardas() const { return Vardas_; }
00048
00053         std::string getPavarde() const { return Pavarde_; }
00054
00055
00056
00061         void setVardas(const std::string& vardas) { Vardas_ = vardas; }
00062
00067         void setPavarde(const std::string& pavarde) { Pavarde_ = pavarde; }
00068
00069
00070
00078         virtual void print() const = 0;
00079
00080
00081
00088         virtual ~Zmogus() {
00089             Vardas_ = "";
00090             Pavarde_ = "";
00091         }
00092
00097         Zmogus(const Zmogus& other)
00098             : Vardas_(other.Vardas_), Pavarde_(other.Pavarde_) {}
00099
00105         Zmogus& operator=(const Zmogus& other) {
00106             if (this != &other) {
00107                 Vardas_ = other.Vardas_;
00108                 Pavarde_ = other.Pavarde_;
00109             }
00110             return *this;
00111         }

```

```

00112
00117     Zmogus(Zmogus&& other) noexcept
00118         : Vardas_(std::move(other.Vardas_)), Pavarde_(std::move(other.Pavarde_)) {}
00119
00125     Zmogus& operator=(Zmogus&& other) noexcept {
00126         if (this != &other) {
00127             Vardas_ = std::move(other.Vardas_);
00128             Pavarde_ = std::move(other.Pavarde_);
00129         }
00130         return *this;
00131     }
00132 };
00133
00134
00135
00151 class Studentas : public Zmogus {
00152 private:
00153     vector<int> nd_rez_;
00154     int egz_rez_ = 0;
00155     double vidurkis_ = 0.0;
00156     double mediana_ = 0.0;
00157     double galutinis_vid_ = 0.0;
00158     double galutinis_med_ = 0.0;
00159
00160 public:
00161
00162
00166     Studentas()
00167         : Zmogus(), nd_rez_(), egz_rez_(0), vidurkis_(0.0), mediana_(0.0),
00168           galutinis_vid_(0.0), galutinis_med_(0.0) {}
00169
00181     Studentas(const std::string& vardas, const string& pavarde, const vector<int>& nd,
00182               const int& egz, const double& vid, const double& med,
00183               const double& gv, const double& gm)
00184         : Zmogus(vardas, pavarde), nd_rez_(nd), egz_rez_(egz), vidurkis_(vid),
00185           mediana_(med), galutinis_vid_(gv), galutinis_med_(gm) {}
00186
00192     Studentas(const std::string& vardas, const std::string& pavarde)
00193         : Zmogus(vardas, pavarde) {}
00194
00195
00196
00201     const vector<int>& getNdRez() const { return nd_rez_; }
00202
00209     vector<int>& getNdRez() { return nd_rez_; }
00210
00215     int getEgzRez() const { return egz_rez_; }
00216
00221     double getVidurkis() const { return vidurkis_; }
00222
00227     double getMediana() const { return mediana_; }
00228
00233     double getGalutinisVid() const { return galutinis_vid_; }
00234
00239     double getGalutinisMed() const { return galutinis_med_; }
00240
00241
00242
00247     void setEgzRez(int egz) { egz_rez_ = egz; }
00248
00253     void setVidurkis(double vid) { vidurkis_ = vid; }
00254
00259     void setMediana(double med) { mediana_ = med; }
00260
00265     void setGalutinisVid(double gv) { galutinis_vid_ = gv; }
00266
00271     void setGalutinisMed(double gm) { galutinis_med_ = gm; }
00272
00273
00274
00280     void print() const override {
00281         std::cout << Vardas_ << " " << Pavarde_
00282                   << " | Galutinis pazymys: " << galutinis_vid_ << "\n";
00283     }
00284
00285
00286
00290     ~Studentas() {
00291         nd_rez_.clear();
00292         egz_rez_ = 0;
00293         vidurkis_ = 0.0;
00294         mediana_ = 0.0;
00295         galutinis_vid_ = 0.0;
00296         galutinis_med_ = 0.0;
00297     }
00298
00299
00303     Studentas(const Studentas& other)

```



```

00304         : Zmogus(other), nd_rez_(other.nd_rez_),
00305         egz_rez_(other.egz_rez_), vidurkis_(other.vidurkis_),
00306         mediana_(other.mediana_),
00307         galutinis_vid_(other.galutinis_vid_),
00308         galutinis_med_(other.galutinis_med_) {}
00309
00315     Studentas& operator=(const Studentas& other) {
00316         if (this != &other) {
00317             Zmogus::operator=(other);
00318             nd_rez_ = other.nd_rez_;
00319             egz_rez_ = other.egz_rez_;
00320             vidurkis_ = other.vidurkis_;
00321             mediana_ = other.mediana_;
00322             galutinis_vid_ = other.galutinis_vid_;
00323             galutinis_med_ = other.galutinis_med_;
00324         }
00325         return *this;
00326     }
00327
00332     Studentas(Studentas&& other) noexcept
00333         : Zmogus(std::move(other)), nd_rez_(std::move(other.nd_rez_)),
00334         egz_rez_(other.egz_rez_), vidurkis_(other.vidurkis_),
00335         mediana_(other.mediana_),
00336         galutinis_vid_(other.galutinis_vid_),
00337         galutinis_med_(other.galutinis_med_) {}
00338
00344     Studentas& operator=(Studentas&& other) noexcept {
00345         if (this != &other) {
00346             Zmogus::operator=(std::move(other));
00347             nd_rez_ = std::move(other.nd_rez_);
00348             egz_rez_ = other.egz_rez_;
00349             vidurkis_ = other.vidurkis_;
00350             mediana_ = other.mediana_;
00351             galutinis_vid_ = other.galutinis_vid_;
00352             galutinis_med_ = other.galutinis_med_;
00353         }
00354         return *this;
00355     }
00356 };
00357
00358 #endif

```

5.7 testavimas.h

```

00001 #ifndef TESTAVIMAS_H
00002 #define TESTAVIMAS_H
00003
00004 #include "lib.h"
00005 #include "studentas.h"
00006
00025 template<typename Container>
00026 void nuskaitymas(Container &S, const int& dydis) {
00027     string path = "test_" + std::to_string(dydis) + ".txt";
00028     cout << "Skaitomas failas " << path << "...\\n";
00029     std::ifstream in(path);
00030     if (!in) {
00031         std::cerr << "Klaida! Nepavyko atidaryti failo " << path << "...\\n";
00032         return;
00033     }
00034     string eil;
00035     getline(in, eil); // skip header row
00036     while (getline(in, eil)) {
00037         if (eil.empty()) continue;
00038
00039         std::istringstream ss(eil);
00040         int paz, pazSuma = 0;
00041         Studentas tmp;
00042         string vrd, pvrđ;
00043         ss >> vrd >> pvrđ;
00044         tmp.setVardas(vrd);
00045         tmp.setPavarde(pvrđ);
00046         while (ss >> paz) {
00047             if (paz > 0 && paz <= 10) {
00048                 tmp.getNdRez().push_back(paz);
00049                 pazSuma += paz;
00050             }
00051         }
00052         if (!tmp.getNdRez().empty()) {
00053             tmp.setEgzRez(tmp.getNdRez().back());
00054             tmp.getNdRez().pop_back();
00055         }
00056         tmp.setVidurkis(static_cast<double>(pazSuma - tmp.getEgzRez()))
00057             / static_cast<double>(tmp.getNdRez().size());

```

```

00058         S.push_back(tmp);
00059     }
00060     in.close();
00061 }
00062
00078 template<typename Container>
00079 void skaiciavimai(Container &S) {
00080     for (auto& s : S) {
00081         std::sort(s.getNdRez().begin(), s.getNdRez().end());
00082         if (s.getNdRez().size() % 2 == 0) {
00083             s.setMediana((s.getNdRez()[s.getNdRez().size() / 2]
00084                 + s.getNdRez()[s.getNdRez().size() / 2 - 1]) / 2.0);
00085         } else {
00086             s.setMediana(s.getNdRez()[s.getNdRez().size() / 2]);
00087         }
00088         s.setGalutinisVid(0.4 * s.getVidurkis()
00089             + 0.6 * static_cast<double>(s.getEgzRez()));
00090         s.setGalutinisMed(0.4 * s.getMediana()
00091             + 0.6 * static_cast<double>(s.getEgzRez()));
00092     }
00093     rusiavimas(S, 1, 3);
00094 }
00095
00110 template<typename Container>
00111 void antras_tyrimas() {
00112     int dydziai[] = {1000, 10000, 100000, 1000000, 10000000};
00113     for (const int& dydis : dydziai) {
00114         auto start_nusk = std::chrono::high_resolution_clock::now();
00115         Container S;
00116         nuskaitymas(S, dydis);
00117         auto end_nusk = std::chrono::high_resolution_clock::now();
00118         std::chrono::duration<double> elapsed_nusk = end_nusk - start_nusk;
00119         cout << std::fixed << std::setprecision(6)
00120             << "Failas su " << dydis << " studentu nuskaitytas per "
00121             << elapsed_nusk.count() << " s\n";
00122
00123         auto start_grupe = std::chrono::high_resolution_clock::now();
00124         Container Dundukai, Galvociai;
00125         skaiciavimai(S);
00126         grupavimas_pirm(S, Dundukai, Galvociai);
00127         auto end_grupe = std::chrono::high_resolution_clock::now();
00128         std::chrono::duration<double> elapsed_grupe = end_grupe - start_grupe;
00129         cout << std::fixed << std::setprecision(6)
00130             << dydis << " studentu surusiuoti i dvi grupes per "
00131             << elapsed_grupe.count() << " s\n";
00132
00133         auto start_isv = std::chrono::high_resolution_clock::now();
00134         isvedimas_i_faila(Dundukai, Galvociai);
00135         auto end_isv = std::chrono::high_resolution_clock::now();
00136         std::chrono::duration<double> elapsed_isv = end_isv - start_isv;
00137         cout << std::fixed << std::setprecision(6)
00138             << dydis << " studentu isvesti i skirtingus failus pagal grupes per "
00139             << elapsed_isv.count() << " s\n";
00140         cout << std::fixed << std::setprecision(6)
00141             << "Visas procesas su " << dydis << " studentu uztruko "
00142             << elapsed_nusk.count() + elapsed_grupe.count() + elapsed_isv.count()
00143             << " s\n\n\n";
00144     }
00145 }
00146
00147 #endif

```

Index

- ~Zmogus
 - Zmogus, [8](#)
- getPavarde
 - Zmogus, [9](#)
- getVardas
 - Zmogus, [9](#)
- include/funkcijos.h, [13](#)
- include/funkcijos_temp.h, [14](#)
- include/grupavimas.h, [17](#)
- include/lib.h, [17](#)
- include/rusiavimas.h, [18](#)
- include/studentas.h, [19](#)
- include/testavimas.h, [21](#)
- operator=
 - Zmogus, [9](#), [10](#)
- print
 - Zmogus, [10](#)
- setPavarde
 - Zmogus, [10](#)
- setVardas
 - Zmogus, [11](#)
- Studentas, [7](#)
- Zmogus, [7](#)
 - ~Zmogus, [8](#)
 - getPavarde, [9](#)
 - getVardas, [9](#)
 - operator=, [9](#), [10](#)
 - print, [10](#)
 - setPavarde, [10](#)
 - setVardas, [11](#)
 - Zmogus, [8](#), [9](#)